

Shortest route between Indian cities using search

```

import heapq

graph = {

    "Hyderabad": {
        "Vijayawada": 275,
        "Nagpur": 500,
        "Bengaluru": 570,
        "Chennai": 626
    },

    "Vijayawada": {
        "Hyderabad": 275,
        "Visakhapatnam": 350,
        "Chennai": 450
    },

    "Visakhapatnam": {
        "Vijayawada": 350,
        "Bhubaneswar": 440
    },

    "Bhubaneswar": {
        "Visakhapatnam": 440,
        "Kolkata": 442
    },

    "Kolkata": {
        "Bhubaneswar": 442,
        "Patna": 580
    },

    "Nagpur": {
        "Hyderabad": 500,
        "Bhopal": 350,
        "Raipur": 300
    },

    "Bhopal": {
        "Nagpur": 350,
        "Jaipur": 600,
        "Agra": 590
    },

    "Raipur": {
        "Nagpur": 300,
        "Patna": 600
    },

    "Patna": {
        "Kolkata": 580,
        "Raipur": 600,
        "Delhi": 1000
    },

    "Bengaluru": {
        "Hyderabad": 570,
        "Chennai": 346,
        "Pune": 840
    },

    "Chennai": {
        "Hyderabad": 626,
        "Vijayawada": 450,
        "Bengaluru": 346
    },

    "Pune": {
        "Bengaluru": 840,
        "Mumbai": 150
    },

    "Mumbai": {
        "Pune": 150,
        "Ahmedabad": 530
    },

    "Ahmedabad": {

```

```

    "Mumbai": 530,
    "Jaipur": 660
},

"Jaipur": {
    "Ahmedabad": 660,
    "Bhopal": 600,
    "Delhi": 307
},

"Agra": {
    "Bhopal": 590,
    "Delhi": 240
},

"Delhi": {
    "Jaipur": 307,
    "Agra": 240,
    "Patna": 1000
}
}

```

```

def uniform_cost_search(start, goal):

    priority_queue = [(0, start, [start])]

    visited = set()

    while priority_queue:

        cost, city, path = heapq.heappop(priority_queue)

        if city in visited:
            continue

        visited.add(city)

        if city == goal:

            print("\nRoute Found:")
            print(" -> ".join(path))

            print("Total Distance:", cost, "km")
            print("Cities Expanded:", len(visited))

            return

        for neighbor, distance in graph[city].items():

            if neighbor not in visited:

                heapq.heappush(
                    priority_queue,
                    (
                        cost + distance,
                        neighbor,
                        path + [neighbor]
                    )
                )

    print("No route found")

```

```

start = input("Enter start city: ")
goal = input("Enter goal city: ")

if start in graph and goal in graph:
    uniform_cost_search(start, goal)
else:
    print("Invalid city name")

```

```

Enter start city: Hyderabad
Enter goal city: Bengaluru

```

```

Route Found:
Hyderabad -> Bengaluru
Total Distance: 570 km
Cities Expanded: 4

```

